RollingCode School

GUÍA RÁPIDA: Métodos de Arrays en JavaScript / TypeScript

Cheat Sheet de referencia práctica para el Trabajo Práctico Integrador (CRUD en string[])

1. Operaciones CRUD Básicas (Mutación de Arrays)

`.push(elem1, elem2)`	Agrega uno o más elementos al FINAL del array. Modifica el array original y retorna la nueva longitud.
`.pop()`	Elimina el ÚLTIMO elemento del array. Modifica el array original y retorna el elemento eliminado.
`.unshift(elem1, elem2)`	Agrega uno o más elementos al INICIO del array. Modifica el array original y desplaza los índices.
`.shift()`	Elimina el PRIMER elemento del array. Modifica el array original y retorna el elemento eliminado.
`.splice(inicio, borrarCant, elem1...)`	Método multipropósito para eliminar, insertar o reemplazar elementos en cualquier posición especificada.

2. Recorrido y Transformación (Métodos con Callbacks)

`.forEach((elem, index) => { ... })`	Ejecuta una función callback por cada elemento del array. No retorna un nuevo arreglo (ideal para listar).
`.map((elem, index) => nuevoValor)`	Crea y retorna un NUEVO array con los resultados de aplicar la función callback a cada elemento.
`.filter((elem) => condición)`	Crea y retorna un NUEVO array con todos los elementos que cumplan con la condición dada (retorne true).
`.find((elem) => condición)`	Retorna el PRIMER elemento que cumpla con la condición dada. Si no encuentra ninguno, retorna undefined.
`.findIndex((elem) => condición)`	Retorna el ÍNDICE del primer elemento que cumpla la condición. Si no existe, retorna -1.

3. Ordenamiento, Unificación y Resumen

`.sort(funcionComparacion?)`	Ordena los elementos del array in situ. Por defecto ordena alfabéticamente como strings.
`.reverse()`	Invierte el orden de los elementos del array in situ (el primero pasa a ser el último y viceversa).
`.slice(inicio, fin)`	Retorna una COPIA SUPERFICIAL (backup) de una porción del array sin modificar el array original.
`.concat(array2, array3...)`	Une dos o más arrays y retorna un NUEVO array combinado sin alterar los arrays existentes.
`.reduce((acum, elem) => nuevoAcum, valorInicial)`	Reduce todos los elementos del array a un ÚNICO VALOR acumulado (string, número, etc.).

4. Métodos de Verificación y Condición (Investigación)

`.some((elem) => condición)`	Comprueba si AL MENOS UN elemento del array cumple con la condición dada. Retorna true o false.
`.every((elem) => condición)`	Comprueba si TODOS los elementos del array cumplen con la condición dada. Retorna true o false.

Recordatorio Clave: Recuerda que en este ejercicio integrador la estructura de datos es exclusivamente un array de strings (string[]). No utilices objetos ({}) ni estructuras anidadas complejas.